

INGENIERÍA DE SOFTWARE II

ACTIVIDAD 8

FUNDAMENTOS DEL PROCESO DE PRUEBAS

Análisis de vulnerabilidades web aplicado a ENLYCE

Producto analizado. ENLYCE, ecosistema inmobiliario compuesto por CRM, API y sitio web público.

Samuel Andres Escobar Saldarriaga

Trabajo individual

26 de septiembre de 2026

Medellín, Colombia

Resumen ejecutivo

Este informe documenta la instalación, ejecución y análisis de pruebas de seguridad sobre ENLYCE en un ambiente local y aislado. La evaluación combinó revisión automatizada de dependencias, análisis dinámico de la aplicación y pruebas específicas de lógica de negocio. El resultado principal fue la confirmación de una vulnerabilidad crítica de autorización a nivel de objeto —BOLA/IDOR— en el flujo público de oportunidades de propietarios.

Las herramientas automatizadas no confirmaron inyección SQL, XSS, traversal, XXE, ejecución remota ni inyección de comandos. Sin embargo, una prueba dirigida reveló que una persona que conociera el correo de otra podía recuperar el identificador de su oportunidad y modificar datos del inmueble. Este contraste demuestra por qué una auditoría seria no puede depender de un solo escáner.

Objetivos

- Reproducir el proceso de instalación y prueba de una aplicación web.
- Generar y organizar un reporte técnico de vulnerabilidades.
- Explicar una vulnerabilidad confirmada, su ataque, impacto y corrección.
- Probar una aplicación construida en una tecnología distinta al backend principal.

Alcance y reglas éticas

Las pruebas se realizaron únicamente sobre instancias locales de ENLYCE, con base de datos temporal y registros sintéticos. No se atacaron servicios de terceros, dominios públicos ni información real de clientes. Las solicitudes destructivas se limitaron al ambiente desechable.

Criterio de evidencia. Un hallazgo se considera confirmado solo cuando puede reproducirse y producir un efecto observable. Una alerta automática, por sí sola, no prueba una vulnerabilidad.

Contenido

1. Contexto técnico de ENLYCE
2. Actividad 8.1 — Instalación y verificación
3. Reporte consolidado de vulnerabilidades
- 4–6. Actividad 8.2 — BOLA/IDOR, evidencia y solución
- 7–9. Actividad 8.3 — Next.js, npm audit y OWASP ZAP
10. Inyección SQL y controles adicionales
11. Conclusiones, referencias y trazabilidad

1. Contexto técnico de ENLYCE

ENLYCE es un CRM inmobiliario con un portal público conectado por API. El núcleo usa ASP.NET Core 10, Entity Framework Core y PostgreSQL; el sitio público está construido con Next.js, React y TypeScript. El CRM interno utiliza React con Vite. La arquitectura separa dominio, aplicación, infraestructura y API.

Componente	Tecnología	Función evaluada
API	ASP.NET Core 10 · Minimal APIs	Autenticación, oportunidades, catálogo y formularios
Persistencia	EF Core · PostgreSQL	Acceso a datos y resistencia a inyección SQL
Sitio público	Next.js 16 · React 19 · TypeScript	Captación, navegación y dependencias
CRM	React · Vite	Autorización por roles y operaciones privadas



Figura 1. Sitio público de ENLYCE ejecutándose localmente.

Entorno de prueba

La API principal se verificó en el puerto 5019 y el sitio web en los puertos 3000/3100. Para los ataques se usó una API temporal en el puerto 5029 y una base PostgreSQL aislada. Esta separación evitó contaminar los datos normales de desarrollo.

2. Actividad 8.1 — Instalación y verificación

PREPARACIÓN

Se instalaron y validaron las herramientas necesarias para levantar la aplicación web y ejecutar pruebas de seguridad reproducibles.

1. Abrir una terminal en el repositorio D:\Proyectos\Enlyce.

2. Verificar Node.js, npm, Docker y .NET. Para este ejercicio se confirmó Node 24.16.0, npm 11.13.0 y Docker 29.5.3.
3. Instalar las dependencias del sitio público con npm install.
4. Levantar el sitio Next.js con npm run dev y comprobar la interfaz en http://localhost:3000.
5. Levantar la API y PostgreSQL aislados para que las pruebas activas no alteren datos de uso normal.
6. Ejecutar npm audit sobre las dependencias y OWASP ZAP sobre la especificación OpenAPI.
7. Ejecutar el arnés propio de seguridad para autenticación, autorización, concurrencia y abuso de recursos.

```

Entorno de pruebas — ENLYCE

PS D:\Proyectos\Enlyce> node --version
v24.16.0
PS D:\Proyectos\Enlyce> npm --version
11.13.0
PS D:\Proyectos\Enlyce> docker --version
Docker version 29.5.3, build d1c06ef
PS D:\Proyectos\Enlyce> npm run dev
Next.js 16.3.5 - http://localhost:3000
PS D:\Proyectos\Enlyce> docker run ... zap-api-scan.py -f openapi -a
OWASP ZAP activo contra API aislada en localhost:5029

Entorno local y base PostgreSQL desechable

```

Figura 2. Verificación del entorno y de las herramientas instaladas.

Herramientas

Herramienta	Tipo	Propósito
npm audit	SCA	Compara dependencias con vulnerabilidades conocidas
OWASP ZAP 2.17	DAST	Prueba automáticamente API y sitio en ejecución
Arnés ENLYCE	Prueba dirigida	Valida controles que requieren contexto del negocio
xUnit	Prueba automatizada	Ejecuta regresiones unitarias e integrales

3. Reporte consolidado de vulnerabilidades

La siguiente tabla resume los resultados reproducibles. La prioridad considera confidencialidad, integridad, disponibilidad y facilidad de explotación.

Prioridad	Hallazgo	Evidencia	Estado
Crítica	BOLA/IDOR en enriquecimiento de oportunidad	Lectura del ID y modificación anónima con correo conocido	Confirmado

Prioridad	Hallazgo	Evidencia	Estado
Alta	Sin límite de intentos de inicio de sesión	60 solicitudes inválidas: 60 respuestas 401 y ninguna 429	Confirmado
Alta	Condición de carrera en deduplicación	30 solicitudes simultáneas produjeron múltiples IDs	Confirmado
Alta	Abuso de tamaño y registro de datos	Campo público de 1 MB fue recibido y procesado	Confirmado
Media	Cabeceras defensivas incompletas	Sin CSP, frame-ancestors/X-Frame-Options, nosniff ni Permissions-Policy	Confirmado
Control	JWT manipulado y alg:none	Ambos rechazados con 401	Resiste
Control	Acceso horizontal entre asesores	Operaciones ajenas rechazadas con 403	Resiste
Control	Inyección SQL y familias relacionadas	ZAP no confirmó explotación	Sin evidencia



Site: <http://host.docker.internal:5029>

Generated on Sun, 27 Sept 2026 00:49:29

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	4
Informational	7
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	1
Low	Exceeded High	http://host.docker.internal:5029	Percentage of responses with status code 4xx	98 %
Info	Informational	http://host.docker.internal:5029	Percentage of responses with status code 2xx	16 %
Info	Informational	http://host.docker.internal:5029	Percentage of responses with status code 5xx	1 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with content type application/json	46 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with content type application/problem+json	1 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method DELETE	1 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method GET	40 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method POST	54 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method PUT	4 %
Info	Informational	http://host.docker.internal:5029	Count of total endpoints	271
Info	Informational	http://host.docker.internal:5029	Percentage of slow responses	50 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Figura 3. Resumen del análisis activo de la API con OWASP ZAP.

Lectura correcta del resultado. Cero hallazgos altos en ZAP no significa seguridad total. ZAP evaluó patrones técnicos; la falla crítica dependía de comprender el flujo de negocio.

4. Actividad 8.2 — Vulnerabilidad seleccionada

4.1 BOLA/IDOR: autorización rota a nivel de objeto

OWASP clasifica BOLA como API1:2023. Ocurre cuando una API recibe el identificador de un objeto y ejecuta una acción sin comprobar de forma suficiente que quien hace la solicitud está autorizado sobre ese objeto. El

hecho de usar un GUID no corrige el problema: el identificador puede filtrarse, reutilizarse o llegar por otra respuesta.

4.2 Cómo funciona el ataque en ENLYCE

1. El atacante conoce o adivina el correo de una persona que ya registró una oportunidad.
2. Envía POST /api/leads con ese correo. El sistema detecta el recontacto, pero responde con el ID, nombre, correo, estado y otros datos de la oportunidad existente.
3. Con el ID revelado, envía PUT /api/leads/{id}/owner-details de forma anónima.
4. La API usa la combinación GUID + correo como prueba de identidad. Como ambos datos ya están disponibles, acepta la solicitud y modifica la oportunidad.

```
POST /api/leads -> 201 Created
respuesta: { id, nombre, email, estado, ... }

PUT /api/leads/{id}/owner-details -> 200 OK
cuerpo: { email, city, neighborhood, expectedPrice, ... }
```

4.3 Impacto

- Confidencialidad: revela datos personales y metadatos de una oportunidad ajena.
- Integridad: permite cambiar ciudad, barrio, precio esperado, mensaje y canal de contacto.
- Negocio: contamina el pipeline comercial y puede desviar la atención del asesor.
- Cumplimiento: aumenta el riesgo asociado al tratamiento de datos personales.

Severidad. Crítica. La explotación no requiere una cuenta autenticada y combina exposición de información con modificación de datos.

5. Evidencia y causa raíz



Figura 4. Resultado del arnés: controles resistentes y fallas confirmadas.

5.1 Evidencia en el código

El endpoint de creación es anónimo y devuelve el objeto completo con su ID. Si el correo ya existe, el manejador registra el recontacto y vuelve a entregar los datos guardados.

```
var result = await handler.HandleAsync(command);
return Results.Created($"api/leads/{result.Id}", result);
.AllowAnonymous();

return new CreateLeadResponse(
    saved.Id, saved.Nombre, saved.Email.Value, saved.Estado.ToString(), ...);
```

El endpoint de enriquecimiento también es anónimo y valida únicamente que el correo enviado coincida con el correo de la oportunidad identificada por el GUID.

```
var lead = await leads.GetByIdAsync(command.LeadId);
var email = Email.Create(command.Email);
if (lead is null || lead.Email != email) return null;
lead.EnrichOwnerInquiry(...);
```

La causa raíz no es el uso de GUID. Es tratar dos datos recuperables —ID y correo— como si fueran una credencial secreta, sin sesión, token de continuación ni autorización por objeto.

6. Solución propuesta y verificación

6.1 Diseño seguro

Comportamiento actual	Comportamiento recomendado
Un recontacto devuelve ID y datos existentes	Responder de forma genérica sin revelar si el correo ya existía
El correo y GUID habilitan la edición	Emitir token aleatorio de un solo uso, corto y vinculado al flujo
Endpoint de edición anónimo permanente	Exigir token válido o sesión autorizada y verificar el objeto
Sin límite visible de solicitudes	Aplicar rate limiting por IP, correo normalizado y operación
Deduplicación vulnerable a carreras	Agregar restricción única/idempotencia y manejar conflictos

6.2 Controles concretos

- Devolver 202 Accepted o una respuesta genérica en recontactos; nunca exponer el objeto existente.
- Generar un token criptográficamente aleatorio de al menos 256 bits, guardar solo su hash, expirar en 10–15 minutos y consumirlo una sola vez.
- Vincular el token a la oportunidad y al propósito específico de completar datos.
- Aplicar autorización a nivel de objeto en cada operación que reciba un identificador.
- Registrar intentos fallidos, cambios y contexto técnico sin almacenar campos sensibles completos.
- Agregar límites de tamaño de cuerpo, validación temprana y rate limiting.

6.3 Pruebas de regresión

1. Registrar dos veces el mismo correo y comprobar que la segunda respuesta no revela ID ni datos existentes.
2. Intentar modificar con ID y correo sin token; debe fallar con una respuesta genérica.
3. Comprobar que un token vencido, reutilizado o ligado a otra oportunidad sea rechazado.
4. Enviar una ráfaga controlada y verificar respuestas 429.
5. Ejecutar nuevamente las 400 pruebas .NET y el arnés de seguridad antes del despliegue.

7. Actividad 8.3 — Aplicación en otra tecnología

Para esta evidencia se seleccionó el sitio público de ENLYCE, desarrollado con Next.js, React y TypeScript. Es una tecnología distinta al backend principal, que está construido en C# con ASP.NET Core. La prueba combinó análisis de dependencias y análisis dinámico.

7.1 Instalación del sitio Next.js

1. Instalar Node.js y comprobar la versión con `node --version`.
2. Abrir una terminal en `apps/website/sitio`.
3. Ejecutar `npm install` para restaurar exactamente el árbol de dependencias definido por el proyecto.
4. Ejecutar `npm run dev`.
5. Abrir `http://localhost:3000` y comprobar navegación, catálogo y formularios.

```
cd D:\Proyectos\Enlyce\apps\website\sitio
node --version
npm --version
npm install
npm run dev
```



Figura 5. Verificación funcional del sitio construido con Next.js y TypeScript.

8. Prueba de dependencias con npm audit

npm audit envía una descripción del árbol de dependencias al registro configurado y devuelve vulnerabilidades conocidas con su nivel y posible corrección. En ENLYCE el comando terminó con código 0 y reportó cero vulnerabilidades conocidas en 501 paquetes al momento de la prueba.

```
cd apps\website\sitio
npm audit

resultado: found 0 vulnerabilities
```

Análisis de dependencias — Next.js / TypeScript

```
PS D:\Proyectos\Enlyce\website\sitio> npm audit

found 0 vulnerabilities

Framework: Next.js 16.3.5
Interfaz: React 19.2.8
Lenguaje: TypeScript 5
Resultado: sin vulnerabilidades conocidas en el árbol npm analizado
```

Figura 6. Resultado real de npm audit sobre el sitio público.

Límite de la herramienta. Un resultado limpio solo cubre vulnerabilidades conocidas en dependencias. No evalúa autorización, lógica de negocio, configuración del servidor ni código propio.

Interpretación

El resultado es positivo, pero no suficiente para afirmar que la aplicación es segura. Por eso se complementó con ZAP y con el arnés dirigido. Esta defensa por capas permitió descubrir una falla crítica que no dependía de una biblioteca vulnerable.

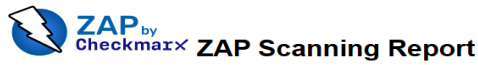
9. Prueba dinámica con OWASP ZAP

ZAP API Scan importa la definición OpenAPI y ejecuta un análisis activo ajustado a APIs. La ejecución de ENLYCE importó 157 URLs. El reporte consolidó 271 puntos de prueba, 119 reglas aprobadas, cero alertas altas, cero medias, cuatro bajas y siete informativas.

9.1 Procedimiento

1. Exponer el documento OpenAPI de la API local.
2. Montar la carpeta de reportes en el contenedor oficial de ZAP.
3. Ejecutar el API Scan contra el ambiente aislado.
4. Revisar manualmente alertas, códigos 5xx y posibles falsos positivos.
5. Cruzar el resultado con pruebas dirigidas de autenticación y autorización.

```
docker run --rm --network host \
-v <reportes>:/zap/wrk/:rw -t zaproxy/zap-stable \
zap-api-scan.py -t http://127.0.0.1:5029/openapi/v1.json \
-f openapi -r zap-api-active.html -J zap-api-active.json
```



Site: <http://host.docker.internal:5029>

Generated on Sun, 27 Sept 2026 00:49:29

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	4
Informational	7
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	1
Low	Exceeded High	http://host.docker.internal:5029	Percentage of responses with status code 4xx	98 %
Info	Informational	http://host.docker.internal:5029	Percentage of responses with status code 2xx	16 %
Info	Informational	http://host.docker.internal:5029	Percentage of responses with status code 5xx	1 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with content type application/json	46 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with content type application/problem+json	1 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method DELETE	1 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method GET	40 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method POST	54 %
Info	Informational	http://host.docker.internal:5029	Percentage of endpoints with method PUT	4 %
Info	Informational	http://host.docker.internal:5029	Count of total endpoints	271
Info	Informational	http://host.docker.internal:5029	Percentage of slow responses	50 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (if highest alert(s) for the step, if any).

Alerts

Figura 7. Reporte generado por ZAP para la API de ENLYCE.

9.2 Hallazgos de configuración

El análisis pasivo del sitio identificó ausencia de Content-Security-Policy, protección contra framing, X-Content-Type-Options: nosniff y Permissions-Policy. También observó exposición de X-Powered-By. Estas fallas no equivalen a ejecución inmediata, pero reducen la defensa en profundidad.

10. Pruebas de inyección SQL y controles adicionales

10.1 Inyección SQL

Sí se probaron ataques de inyección SQL. ZAP ejecutó reglas genéricas y específicas de PostgreSQL, incluidas variantes basadas en error, booleanos y tiempo. No se confirmó inyección SQL en los endpoints evaluados. El uso habitual de LINQ y EF Core reduce el riesgo porque los valores se parametrizan; aun así, cualquier uso futuro de SQL crudo debe revisarse.

Familia de ataque	Resultado	Observación
SQL Injection	Sin confirmación	No hubo respuesta que demostrara ejecución de SQL inyectado
PostgreSQL time-based	Sin confirmación	No se observó retraso atribuible al payload
XSS	Sin confirmación	No se confirmó ejecución de script
Path traversal / XXE	Sin confirmación	No se obtuvo lectura de archivo ni expansión de entidad
RCE / command injection	Sin confirmación	No hubo evidencia de ejecución en el servidor

10.2 Controles que resistieron

- Los JWT con firma alterada y con alg:none fueron rechazados con 401.

- Un asesor no pudo consultar, mover ni reasignar oportunidades ajenas; las respuestas fueron 403.
- Un origen CORS no autorizado no recibió permiso de lectura.
- La sobrescritura del método HTTP fue rechazada con 405.

10.3 Precisión del resultado

“Sin confirmación” no significa “imposible”. Significa que, con el alcance, entradas y tiempo de esta ejecución, no se obtuvo evidencia reproducible. La conclusión debe revisarse después de cada cambio en consultas, dependencias o endpoints.

11. Conclusiones y plan de mejora

La práctica demostró que las pruebas de seguridad deben combinar herramientas y criterio. npm audit confirmó un árbol de dependencias sin vulnerabilidades conocidas; ZAP cubrió patrones técnicos y configuraciones; el arnés dirigido encontró el riesgo más grave al modelar el comportamiento de una persona atacante dentro del flujo real de propietarios.

Prioridad inmediata

1. Cerrar BOLA/IDOR: respuesta pública opaca, token de continuación y autorización por objeto.
2. Incorporar rate limiting en autenticación y formularios públicos.
3. Corregir deduplicación concurrente con restricción de base de datos e idempotencia.
4. Aplicar límites de tamaño, redacción de logs y cabeceras defensivas.
5. Ejecutar xUnit, npm audit, ZAP y el arnés en integración continua.

Conclusión central. La ausencia de alertas críticas en un escáner no prueba ausencia de vulnerabilidades. La autorización debe probarse con escenarios del negocio y con datos aislados.

Resultado frente a la guía

Punto	Evidencia entregada
8.1	Instalación, herramientas, procedimiento y reporte consolidado
8.2	Definición, ataque, causa raíz, impacto, solución y regresión de BOLA/IDOR
8.3	Instalación y prueba del sitio Next.js/TypeScript con capturas reales

Referencias

OWASP Foundation. (2023). API1:2023 Broken Object Level Authorization. <https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/>

ZAP Project. (2026). ZAP API Scan. <https://www.zaproxy.org/docs/docker/api-scan/>

npm, Inc. (2026). npm-audit — npm Docs. <https://docs.npmjs.com/cli/audit.html/>

Microsoft. (2026). SQL Queries — Entity Framework Core. <https://learn.microsoft.com/en-us/ef/core/querying/sql-queries>

Trazabilidad de evidencias

Evidencia	Origen
Informe de auditoría	docs/seguridad/auditoria-local-2026-09-26.md
Arnés reproducible	tools/security/strong-audit.mjs
Reporte ZAP	security-reports/zap-api-active.html
Pruebas .NET	400 de 400 aprobadas
Auditoría npm	npm audit: 0 vulnerabilidades

Anexo — Criterios de aceptación

- No divulgar identificadores ni datos de oportunidades preexistentes en flujos públicos.
- Toda mutación de un objeto debe validar identidad, propósito y autorización.
- Las ráfagas deben producir 429 antes de consumir recursos excesivos.
- Las pruebas de seguridad deben ejecutarse sobre datos sintéticos y dejar evidencia reproducible.

Fin del informe.